JUnit

Topic : **Introduction to JUnit**

JUnit 5

UNIT TESTING

- Unit Testing refers to the testing of small chunks of codes
- It helps in early identification of defects
- The developers are tend to spend more time on reading the code
- Successful code increases the confidence of the developers

What is Unit Testing?

- A Unit Test is a piece of code written by developer to test certain functionality of code.
- The class which we want to test is called as System/Class Under Test.
- The method which we want to test is called as Method Under Test.
- Unit test instantiates class under test, executes method under test and verifies that it worked as expected.

What makes a Good Unit Test?

- Easy to write
- Readable
- Reliable
- Fast
- Isolated

Two types of Unit Testing

Manual Testing	Automated Testing
• If the test cases are tested manually without any automated tool, it is manual testing.	• If the test cases are tested by tool support, it is automated testing.
• It is less reliable and time-consuming	• It is more reliable and faster

Reflection Spot

Java Unit Testing with JUnit 5

Introduction to JUnit

JUnit 5

What is JUnit?

- Junit is a Unit Testing framework for Java
- The framework is used by Java Developers to write and execute tests
- Every time a new code is added, all the test cases have to be re-executed

What's JUnit ?

JUnit is a simple, open source framework to write and run repeatable tests.

junit.org

JUnit Annotations

- JUnit Annotations refer to the syntactic meta-data added to the Java source code for better structure and readability
- The biggest difference between JUnit4 and JUnit3 is the introduction of Junit Annotations

Some important and frequently used JUnit Annotations

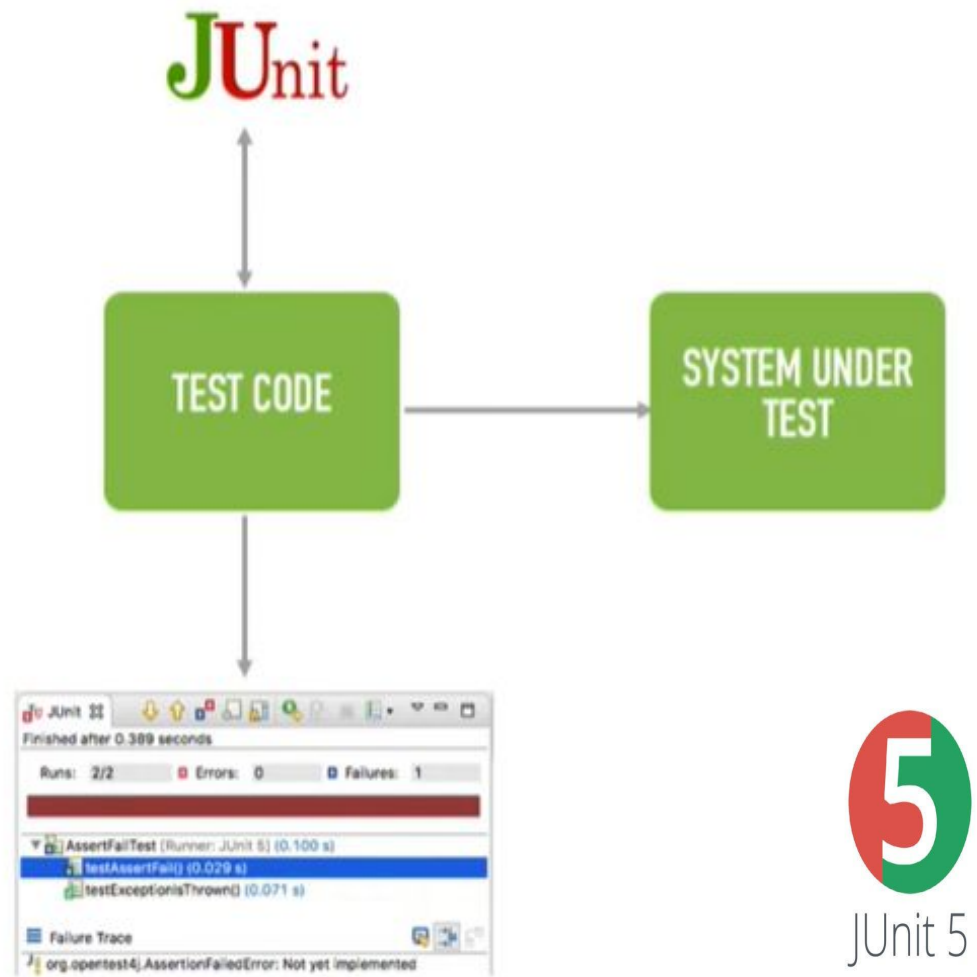
@Test	Tells JUnit which public void method can be run as a test case
@Before	To execute some statement before each test case
@After	To execute some statement after each test case
@Ignores	Used to ignore some statements during test execution

Some important and frequently used JUnit Annotations

@BeforeClass	Used to execute a statement before all the test cases
@AfterClass	Used to execute a statement after all the test cases
@Test (time out =500)	Used to set some timeout while executing the test
@Test (expected=Illegal ArgumentException.class)	Used to handle some exception during test execution

How JUnit Works?

- JUnit is a Java testing framework used to test a piece of Java code (Code Under Test or System Under Test).
- It has a Runner which determines the tests, executes those tests, reports the results.
- JUnit test validates expected result with actual output from the code under test. Thus, based on assertions success or failure it provides feedback to developer.



JUnit 5

JUnit History

JUnit 5

1997

Kent Beck and Erich Gamma released first of JUnit

2000

JUnit website got launched. junit.org

2006

JUnit 4 got released.

2015

Crowdfunding campaign for **junit-lambda** project started

2017

JUnit 5 got released.

JUnit 5

Problem with JUnit 4

junit.jar

- A single fat jar
- No flexible API
- No Java 8 features
- No Extension points

Solution with JUnit 5

JUnit 5

- Multiple small jars
- Flexible API
- Java 8 features
- Extension

JUnit 5

Separations of Concerns (JUnit 5)

- An API for developers to write tests
- Mechanism to discover and execute tests
- An API for external tools to run tests in their environment

JUnit 5

JUnit 5 Components

JUnit 5

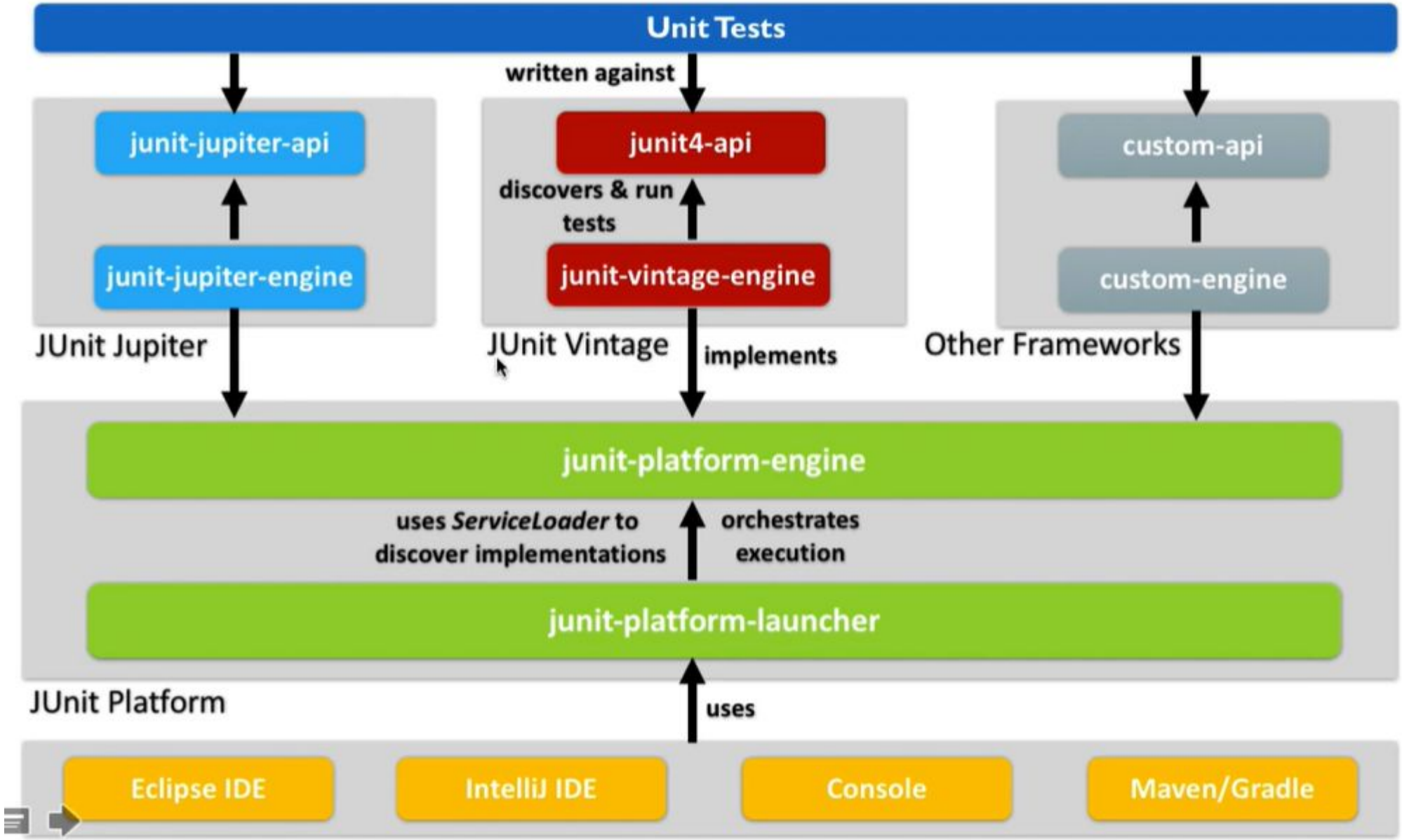
- JUnit 5 is majorly divided into 3 main components.
- JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage

JUnit Platform

JUnit Jupiter

JUnit Vintage

JUnit 5 Architecture



JUnit Installation

- Step 1:install java and set path
- Step 2:install eclipse Ide
- Step 3: create project
- Step 4: add jar files or select /in build jar files

Java Unit Testing with JUnit 5

Building Your First JUnit 5 Test

JUnit 5

```
package com.hubberspot.junit5;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class FirstJunitTest {

	@Test
	void test() {
		fail("Not yet implemented");// remove and run
	}

}
```

Java Unit Testing with JUnit 5

JUnit 5 - @Test Annotation

JUnit 5

@Test annotation in JUnit 5

- @Test annotation is applied over methods to mark them as test methods.
- It is present in org.junit.jupiter.api package.
- Visibility of @Test annotated method can be made public, default and protected.

Execution steps

- **Step 1:** create new- project –name- Junit- finish
- **Step 2:** Right click on project-build path- source folder- name- Test
- **Step 3:** right click on TEST FOLDER-NEW- Junit Test Case
- Select junit Jupiter 5
- Package name- `com.hubberspot.junit5`
- Name: `FirstJUnitProject`
- Select – perform jar files attached
- **Step 4:create new file**
- **Right click on TEST-Name- testannotation**
- Package name- `com.hubberspot.junit5`
- Name: `FirstJUnitProject`
- Select – perform jar files attached

```java
package com.hubberspot.junit5;


import static org.junit.jupiter.api.Assertions.assertEquals;


import org.junit.jupiter.api.Test;


public class TestAnnotation {
@Test
public void test() {
assertEquals(2,2);  // change value and test not equal 3,2 so failed
}


}
```

JUnit 5

Execution steps

- **Step 1:** create new- project –name- Junit- finish
- **Step 2:** Right click on src folder- name- Testclassproject-finish
- **Step 3: open** Testclassproject file and add string comparison code

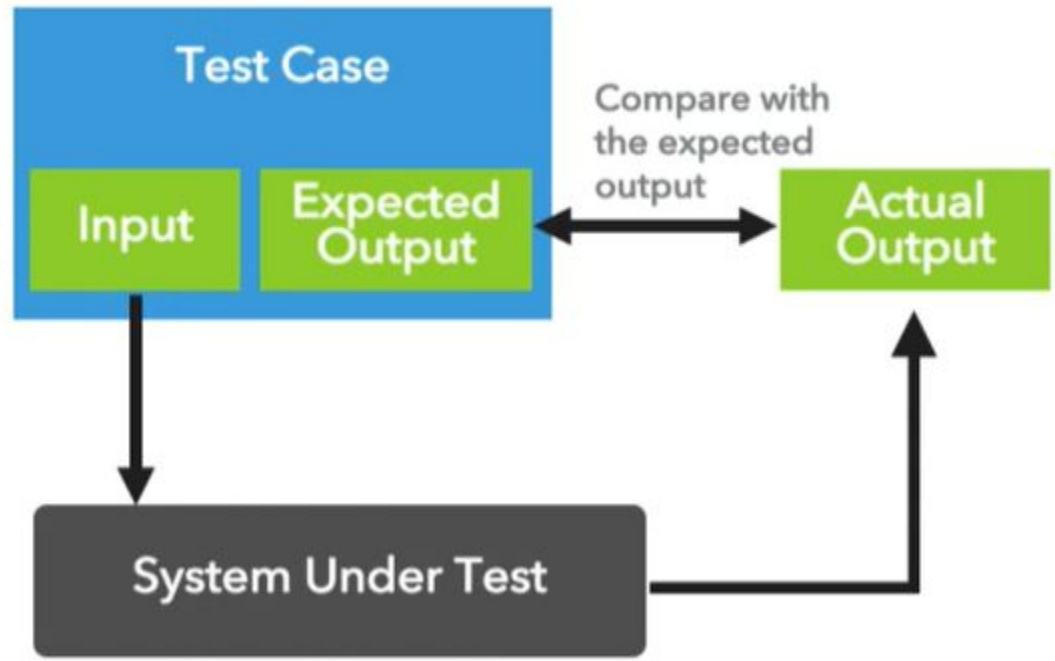
- **COMPARISION OF STRING**
- import static org.junit.Assert.assertEquals;
- import org.junit.After;
- import org.junit. Before;
- import org.junit.Test;

- public class TestJunit {
- @Test // add junit4
- public void TestJunit() {
- System.out.println("Let's check this string");
- String str1= "Let's check this string";
- assertEquals("Let's check this string", str1);
- }

- }

Assertions in JUnit 5

- JUnit 5 assertions help us in validating the expected output with the actual output of a test case.
- Assertions are static methods that we call in our tests to verify expected behavior.
- JUnit Jupiter assertions are present in org.junit.jupiter.Assertions class.
- Assertions methods supports features of Java 8 such as lambda expressions, streams etc.



JUnit 5

Thank you!